

STA141B-HW2

Stella Liu

2026-04-29

Assignment 2: Reading Computer Log files

```
## Warning in readLines("MergedAuth.log"): incomplete final line found on
## 'MergedAuth.log'
```

```
[1] "Nov 30 06:47:01 ip-172-31-27-153 CRON[22087]: pam_unix(cron:session): session opened for user root
by (uid=0)" [1] "Nov 30 06:47:03 ip-172-31-27-153 CRON[22087]: pam_unix(cron:session): session closed
for user root"
```

Find which lines match this general description of the metadata and also characterize the lines that don't match. Be as inclusive as makes sense.

Create a data.frame with a row for each actual log message line, with the variables

- date-time, • the name of the host collecting the log messages, • the app(lication), • the process identifier (PID), • the message, i.e, the remainder of the log line, • the name of the log file from which the line came, e.g. auth.log.

```
[1] ""
[2] "Mar 27 13:08:09 ip-10-77-20-248 systemd: pam_unix(systemd-user:session): session opened for user
ubuntu by (uid=0)"
[3] "Mar 27 13:09:37 ip-10-77-20-248 sudo: ubuntu : TTY=pts/0 ; PWD=/home/ubuntu ; USER=root
; COMMAND=/usr/bin/curl -L -O https://artifacts.elastic.co/downloads/beats/filebeat/filebeat-5.2.2-
amd64.deb"
```

Use general description: timestamp, host, app, PID, and message to match lines.

Lines that did not match include empty entries and ones with different structures and different formats. For example second line has PID systemd: instead of in the usual bracket format, pam_unix() also contains different content. Similarly, the third line is in sudo log format, also structurally different.

When we come across lines that don't fit this format, we characterize them as systemd entries and sudo logs instead of ignoring them, collecting them as a separate data structure and extracting directly from the raw string later on.

Data Validation and Exploration

- How many log files are there? What are their names?

```
[1] 5 [1] "auth.log" "auth2.log"
[3] "loghub/Linux/Linux_2k.log" "loghub/Mac/Mac_2k.log"
[5] "loghub/OpenSSH/SSH_2k.log"
```

There are 5 non-NA log files, as listed above they are "auth.log", "auth2.log", "loghub/Linux/Linux_2k.log", "loghub/Mac/Mac_2k.log", and "loghub/OpenSSH/SSH_2k.log".

- Verify that the PIDs are all numbers.

[1] TRUE

- How many lines are in each log file?

	auth.log	auth2.log	loghub/Linux/Linux_2k.log
	86840	7122	2000
loghub/Mac/Mac_2k.log			
	2000	2000	
	Host name	Log number	

```
41 ip-172-31-27-153 86839 40 ip-10-77-20-248 6326 42 LabSZ 2000 39 combo 1849 3 authorMacBook-Pro 543
20 calvisitor-10-105-162-105 334
```

- What are the range of date-times for the messages? for each of the different log files in the combined file? How many days does each log file span?

```
df_finallogfile_full : auth.log[1]"2026 - 11 - 3006 : 39 : 00PST""2026 - 12 - 3122 : 27 : 48PST" - - - - -
- - - - -df_finallog_file_full: auth2.log [1] "2026-03-27 13:06:56 PDT" "2026-04-20 14:14:29 PDT" - - - - -
- - - - -df_finallogfile_full : loghub/Linux/Linux2k.log[1]"2026 - 06 - 1415 :
16 : 01PDT""2026 - 07 - 2714 : 41 : 59PDT" - - - - -
- - - - -df_finallog_file_full:
loghub/Mac/Mac_2k.log [1] "2026-07-01 09:00:55 PDT" "2026-07-08 08:10:46 PDT" - - - - -
- - - - -df_finallogfile_full : loghub/OpenSSH/SSH2k.log[1]"2026 - 12 - 1006 : 55 :
46PST""2026 - 12 - 1011 : 04 : 45PST"df_finallog_file_full: auth.log [1] 31.65889 - - - - -
- - - - -df_finallogfile_full : auth2.log[1]24.04691 - - - - -
- - - - -df_finallog_file_full:
loghub/Linux/Linux_2k.log [1] 42.97637 - - - - -df_finallogfile_full :
loghub/Mac/Mac2k.log[1]6.965174 - - - - -
- - - - -df_finallog_file_full: loghub/OpenSSH/SSH_2k.log
[1] 0.1729051
```

We unify the units to be in days instead of including values in both days and hours.

- Do the application names contain numbers? If so, are they just versions, e.g. ssh2, or is there additional structure to the numbers?

[1] "BezelServices 255.10"

Application names does contain numbers, for example "BezelServices 255.10" has element 255.10 that is a version structure.

- Is the host value constant/the same for all records in each log file?

```
df_finallogfile_full : auth.log[1]2 - - - - -
- - - - -df_finallog_file_full: auth2.log [1] 2 - - - - -
- - - - -df_finallogfile_full : loghub/Linux/Linux2k.log[1]2 - - - - -
- - - - -
- - - - -df_finallog_file_full: loghub/Mac/Mac_2k.log [1] 39 - - - - -
df_final$log_file_full: loghub/OpenSSH/SSH_2k.log [1] 1
```

The host value is not constant in each log file. In every log file besides SSH_2k.log, there are multiple machines with unique host numbers being more than 1.

- What are the most common apps (daemons/programs) that are logging information on each of the different hosts?

sshd	CRON	ftpd	kernel	sshd(pam_unix)
91341	2857	916	775	677

The top 5 most common apps are sshd, CRON, ftpd, kernel, and sshd(pam_unix). The most common app is sshd with 91,341 unique entries.

```
airbears2-10-142-108-38 airbears2-10-142-110-255 authorMacBook-Pro 9 11 35 calvisitor-10-105-160-179
calvisitor-10-105-160-181 calvisitor-10-105-160-184 10 2 14 calvisitor-10-105-160-205 calvisitor-10-105-
160-210 calvisitor-10-105-160-22 8 3 3 calvisitor-10-105-160-226 calvisitor-10-105-160-237 calvisitor-10-
105-160-37 7 14 2 calvisitor-10-105-160-47 calvisitor-10-105-160-85 calvisitor-10-105-160-95 2 24 19
calvisitor-10-105-161-176 calvisitor-10-105-161-225 calvisitor-10-105-161-231 3 9 1 calvisitor-10-105-161-77
calvisitor-10-105-162-105 calvisitor-10-105-162-107 1 33 6 calvisitor-10-105-162-108 calvisitor-10-105-162-124
calvisitor-10-105-162-138 3 11 1 calvisitor-10-105-162-175 calvisitor-10-105-162-178 calvisitor-10-105-162-211
2 35 2 calvisitor-10-105-162-228 calvisitor-10-105-162-32 calvisitor-10-105-162-81 2 6 2 calvisitor-10-105-162-
98 calvisitor-10-105-163-10 calvisitor-10-105-163-147 2 8 2 calvisitor-10-105-163-168 calvisitor-10-105-163-202
calvisitor-10-105-163-253 1 26 13 calvisitor-10-105-163-28 calvisitor-10-105-163-9 combo 1 3 16 ip-10-77-20-
248 ip-172-31-27-153 LabSZ 7 2 1
```

- How many unique logging hosts look like IP addresses? List the most common ones.

```
[1] 2 ip_hosts_all ip-172-31-27-153 ip-10-77-20-248 86839 6326
```

There are 2 unique logging hosts that look like IP addresses, most common ones being ip-172-31-27-153 and ip-10-77-20-248.

Logins - valid and invalid

- Find all valid/successful logins – What are the user names and Internet Protocol (IP) addresses of those logins
- Find the invalid user login/ids – What were the associated IP addresses? – Were there multiple invalid user logins from the same IP addresses? – Were there valid logins from those IP addresses? – Are there multiple IPs using the same invalid login?

ip_bad Freq

```
1 1.189.205.173 2 2 1.234.21.115 12 3 1.246.219.50 2 4 1.30.211.144 7 5 101.226.249.53 5 6 101.231.72.111 1
user_bad Freq 1 ————— 1 2 \026 2 3 0 5 4 0000 2 5 010101 2 6 1111 2 ip_bad 188.87.35.25 218.25.17.234
220.99.93.50 61.197.203.243 123.57.51.31 409 409 409 360
```

5 most frequent invalid IP's are the above listed.

```
[1] "85.245.107.41" "127.0.0.1" "24.151.103.17"
```

After checking, there were overlaps between valid and invalid IPs, specifically at "85.245.107.41" "127.0.0.1" and "24.151.103.17", returning TRUE for the overlap check, appearing in both. Thus we conclude there were valid logins from those invalid IP addresses.

- Are these related IPs, e.g., from the same network/domain?

We notice that IP addresses that are in similar cluster ranges experience multiple repeated unauthorized access attempts, suggesting that these related IPs are from the same network, related by network proximity.

```
[1] "122.176.37.221" "95.152.57.58" "90.144.183.19" "186.128.152.44" [5] "201.177.23.130" "190.178.62.6"
"190.178.62.6" "190.178.62.6"
[9] "190.178.62.6" "190.178.62.6" "201.43.243.37" "68.33.123.70"
[13] "190.49.42.132" "112.135.124.229" "62.73.115.98" "2.60.103.231"
[17] "31.162.29.148" "179.39.18.38" "115.209.121.31" "113.89.184.179" [21] "191.85.133.91" "191.85.133.91"
"191.85.133.91" "191.85.133.91"
[25] "111.0.82.180" "31.162.4.186" "5.37.195.14" "118.180.18.102" [29] "5.144.9.33" "79.165.2.209"
```

"14.185.87.49" "190.96.200.229" [33] "5.74.204.136" "5.74.204.136" "186.133.159.181" "186.133.159.181"
 [37] "186.133.159.181" "186.133.159.181" "186.133.159.181" "218.60.136.106" [41] "58.19.145.242"
 "117.192.138.116" "112.237.146.110" "179.37.3.130"
 [45] "217.100.114.122" "39.71.33.50" "115.211.146.244" "221.122.101.203" [49] "190.97.81.95" "113.124.141.181"
 "5.141.40.99" "143.208.24.234" [53] "115.58.179.206" "201.178.186.182" "123.96.41.232" "123.115.51.110"
 [57] "182.32.215.94" "181.25.206.27" "181.25.206.27" "218.60.136.106" [61] "106.57.58.19" "181.23.168.176"
 "181.25.201.155" "181.25.201.155" [65] "111.40.168.90" "122.189.198.238" "60.187.118.40" "122.191.89.89"
 [69] "42.184.142.151" "82.64.2.59" "114.32.100.101" "122.244.28.82"
 [73] "123.153.146.183" "181.26.186.35" "181.26.186.35" "181.26.186.35"
 [77] "181.26.186.35" "181.26.186.35" "183.152.79.79" "1.30.211.144"
 [81] "5.167.75.191" "93.120.176.237" "78.106.21.86" "112.251.168.248" [85] "119.193.140.176" "58.19.144.50"
 "122.112.235.133" "122.189.193.214" [89] "49.4.143.105" "49.4.143.105" "49.4.143.105" "49.4.143.105"
 [93] "49.4.143.105" "49.4.143.105" "49.4.143.105" "49.4.143.105"
 [97] "49.4.143.105" "49.4.143.105" "49.4.143.105" "49.4.143.105"
 [101] "49.4.143.105" "49.4.143.105" "49.4.143.105" "49.4.143.105"
 [105] "49.4.143.105" "49.4.143.105" "49.4.143.105" "49.4.143.105"
 [109] "61.166.73.66" "122.191.88.115" "186.130.83.53" "123.96.5.168"
 [113] "111.40.30.206" "188.18.252.218" "125.107.136.165" "190.107.182.33" [117] "112.101.164.200"
 "223.244.185.76" "182.243.87.6" "91.243.236.123" [121] "49.4.143.5" "222.89.76.13" "191.81.42.216"
 "183.93.215.158" [125] "122.190.143.18" "61.183.117.250" "123.120.200.51" "73.231.4.205"
 [129] "110.78.174.75" "49.4.143.181" "186.128.141.232" "46.89.129.145"
 [133] "59.174.52.12" "103.230.120.26" "122.189.197.241" "123.119.111.172" [137] "123.164.142.82"
 "175.162.187.121" "60.165.208.28" "218.91.34.237"
 [141] "49.84.87.84" "222.187.86.51" "77.231.252.103" "95.190.198.34"
 [145] "94.154.25.149" "183.146.159.20" "126.59.251.31" "183.93.253.159" [149] "219.82.145.223" "122.163.61.218"
 "122.163.61.218" "122.163.61.218" [153] "122.163.61.218" "122.163.61.218" "14.54.210.101" "14.54.210.101"
 [157] "105.101.221.33" "37.110.24.51" "181.25.189.115" "178.219.248.139" [161] "177.38.145.209"
 "191.83.152.32" "170.79.155.119" "46.30.160.83"
 [165] "201.178.245.106" "181.211.173.182" "122.144.136.83" "1.189.205.173"
 [169] "181.23.26.185" "201.27.216.125" "179.208.151.103" "119.193.140.203" [173] "182.243.85.75"
 "61.174.116.31" "111.40.166.130" "186.129.147.223" [177] "37.78.105.176" "58.100.135.31" "178.161.33.80"
 "179.38.76.250"
 [181] "186.47.222.98" "201.178.81.113" "201.178.81.113" "201.178.81.113" [185] "201.178.81.113"
 "201.178.81.113" "201.178.81.113" "68.182.39.76"
 [189] "122.5.240.60"

- What IP address had too many authentication failures?

ip_bad 61.197.203.243 409

The most authentication failures occurs at 61.197.203.243, happening 409 times.

Sudo commands

- What are the executables/programs run via sudo – By what user – What machine are they on?

user	machine
------	---------

1	ubuntu	auth2.log	4	ubuntu	auth2.log	7	ubuntu	auth2.log	10	ubuntu	auth2.log	13	ubuntu	auth2.log	16		
ubuntu	auth2.log	executable	1	/usr/bin/curl	-L	-O	https://artifacts.elastic.co/downloads/beats/filebeat/	filebeat-5.2.2-amd64.deb	4	/usr/bin/apt-key	add	-	7	/usr/bin/apt-get	install	apt-transport-https	10
/usr/bin/tee	-a	/etc/apt/sources.list.d/elastic-5.x.list	13	/usr/bin/apt-get	update	16	/usr/bin/apt-get	install	filebeat								

We choose to get data directly from the raw strings for accuracy since there is data structure difference between sudo logs and standard logs.

Appendix: R code

```
knitr::opts_chunk$set(echo = TRUE)
set.seed(1)
computer_log = readLines("MergedAuth.log")
# check to understand data strcture
computer_log[4]
computer_log[5]
#line begins at #, identify headers and extract
is_header <- grepl("^#", computer_log)
log_names <- sub("#\\s*", "", computer_log[is_header])

#grouping ID, cumsum for each header
log_id <- cumsum(is_header)

#NA vector, fill in for valids
log_file_full <- rep(NA_character_, length(computer_log))

#map those greater than 0
has_header_assigned <- log_id > 0
log_file_full[has_header_assigned] <- log_names[log_id[has_header_assigned]]

#create dataframe
df_raw <- data.frame(
  line = computer_log,
  log_source = log_file_full,
  stringsAsFactors = FALSE
)

df_clean <- df_raw[!is_header, ]
#parse: timestamp, host, app, PID, message
pattern <- "^[A-Z][a-z]{2}+[0-9]{1,2}+[0-9:]{8})+([^\ ]+)+([^\ ]+)[\ ]+([0-9+)[\ ]+): (.*)$"
data_lines <- df_clean$line

#extract groups
m <- regexec(pattern, data_lines)
matches <- regmatches(data_lines, m)

#build data frame, check length for no matches
df <- do.call(rbind, lapply(matches, function(x) {
  if(length(x) == 6) return(x[-1])
  else return(rep(NA, 5))
}))

#combine into dataframe
df_final <- as.data.frame(df, stringsAsFactors = FALSE)
colnames(df_final) <- c("datetime", "host", "app", "pid", "message")

df_final$log_file_full <- df_clean$log_source

#lines that did not match
bad_lines <- data_lines[sapply(matches, length) != 6]
bad_lines[2:4]
```

```

#number of unique non NA log files
length(unique(na.omit(df_final$log_file_full)))
unique(na.omit(df_final$log_file_full))
#check non NA PIDs to be numerical
only_pids <- df_final$pid[!is.na(df_final$pid)]
all(grepl("[0-9]+$", only_pids))
#number of lines per log file
table(df_final$log_file_full)
summary_table <- as.data.frame(table(df_final$host))
colnames(summary_table) <- c("Host name", "Log number")
summary_table <- summary_table[order(-summary_table$`Log number`), ]
head(summary_table)
#convert time
df_final$time <- as.POSIXct(trimws(df_final$datetime), format="%b %d %H:%M:%S")

#range
by(df_final$time, df_final$log_file_full, range, na.rm = TRUE)

#day
by(df_final$time, df_final$log_file_full, function(x) {
  r <- range(x, na.rm = TRUE)
  as.numeric(diff(r), units = "days")
})
#app containing numbers
apps_with_nums <- df_final$app[grepl("[0-9]", df_final$app)]
unique(apps_with_nums)
#host consistency per log
by(df_final$host, df_final$log_file_full, function(x) length(unique(x)))
#top 5 most common
sort(table(df_final$app), decreasing = TRUE)[1:5]
#unique app per host
tapply(df_final$app, df_final$host, function(x) length(unique(x)))
ip_like_regex <- "ip-[0-9]{1,3}-[0-9]{1,3}-[0-9]{1,3}-[0-9]{1,3}"

ip_hosts_all <- df_final$host[grepl(ip_like_regex, df_final$host)]
unique_ip_hosts <- unique(ip_hosts_all)

length(unique_ip_hosts)
sort(table(ip_hosts_all), decreasing = TRUE)
#valid
valid_logins <- df_final[grepl("Accepted", df_final$message), ]
user <- sub(".*for ([^ ]+) from.*", "\\1", valid_logins$message)
ip <- sub(".*from ([0-9\\.]+).*", "\\1", valid_logins$message)

#invalid
invalid_logins <- df_final[grepl("Invalid user|Failed password", df_final$message), ]
user_bad <- sub(".*Invalid user ([^ ]+) from.*", "\\1", invalid_logins$message)
ip_bad <- sub(".*from ([0-9\\.]+).*", "\\1", invalid_logins$message)

#extract IP
ip_in_msg_regex <- "\\d{1,3}(\\.\\d{1,3}){3}"
invalid_ips <- regmatches(invalid_logins$message, regexpr(ip_in_msg_regex, invalid_logins$message))

```

```

#invalid from same IP
ip_summary <- as.data.frame(table(ip_bad))
head(ip_summary)

#invalid across IPs
user_summary <- as.data.frame(table(user_bad))
head(user_summary)

#5 most invalid
sort(table(ip_bad), decreasing = TRUE)[1:5]
#check overlap, return IP address that gives TRUE
unique(ip_bad[ip_bad %in% ip])
#extract IP
fail <- df_final[grepl("maximum authentication attempts exceeded", df_final$message), ]
(sub(".*from ([0-9\\.]+).*", "\\1", fail$message))
#most authentication failures
tail(sort(table(ip_bad)), 1)
#find sudo rows
sudo_idx <- grepl("sudo:", df_clean$line)
raw_sudo_lines <- df_clean$line[sudo_idx]

#extract from raw, format sudo pattern
sudo_user <- sub(".*sudo:\\s+([~:]+).*", "\\1", raw_sudo_lines)
sudo_mach <- df_clean$log_source[sudo_idx]
sudo_exec <- sub(".*COMMAND=(.*)", "\\1", raw_sudo_lines)

sudo_results <- data.frame(
  user = sudo_user,
  machine = sudo_mach,
  executable = sudo_exec,
  stringsAsFactors = FALSE
)

#clean those wo commands
sudo_results <- sudo_results[grepl("/", sudo_results$executable), ]

head(sudo_results)
cat("\\\\newpage") #new page

```